

CONFIGURING STORAGE WITH ANSIBLE MODULES

Red Hat Ansible Engine provides a collection of modules to configure storage devices on managed hosts. Those modules support partitioning devices, creating logical volumes, and creating and mounting filesystems.

The parted Module

The `parted` module supports the partition of block devices. This module includes the functionality of the `parted` command, and allows to create partitions with a specific size, flag, and alignment.

The following table lists some of the parameters for the parted module.

Partition Management Parameters

Parameter	Description
align	Defines how partitions are aligned on the disk. Common values include none, cylinder, or optimal. Proper alignment improves performance on SSDs and modern disks.
device	Specifies the target block device (e.g., /dev/sda, /dev/vdb). This is the disk where partitions will be created or modified.
flags	Sets partition flags such as boot, lvm, raid, or swap. These flags tell the system how the partition should be used.
number	The partition number (e.g., 1, 2, 3). Each partition on a disk has a unique number.
part_end	Defines where the partition ends, measured from the start of the disk. You can specify sizes like 10GB, 100%, or 500MiB.
state	Determines whether the partition should exist. Use present to create or ensure it exists, or absent to remove it.
unit	Sets the measurement unit for partition sizes — e.g., MiB, GiB, TB, or %. This ensures consistent sizing across different disks.

Example Usage

```
``yaml
```

```
- name: Create a 10GB partition
```

```
  ansible.builtin.parted:
```

```
    device: /dev/sdb
```

```
    number: 1
```

```
    state: present
```

```
    part_end: 10GiB
```

```
    unit: GiB
```

```
    flags: [lvm]
```

```
    align: optimal
```

```
``
```

This example creates a **10 GiB LVM partition** on `/dev/sdb` with optimal alignment.

In disk partitioning, **alignment** refers to how partition boundaries are positioned relative to the physical storage blocks on a disk. Proper alignment ensures optimal performance and longevity, especially for SSDs and modern hard drives.

Partition Alignment Explained

Alignment Type	Description	Typical Use Case
none	No special alignment — partitions start wherever space is available. Can cause misalignment and slower performance.	Legacy systems or manual layouts.
cylinder	Aligns partitions to cylinder boundaries (used in older HDD geometry).	Obsolete; rarely used today.
optimal	Aligns partitions to the disk's physical block size (usually 1 MiB boundaries). Maximizes performance and reduces write amplification.	Recommended for SSDs, RAID arrays, and modern disks.

Why Alignment Matters

- Prevents **read/write inefficiency** by matching partition boundaries with physical sectors.
- Avoids **misaligned I/O**, which can double the number of disk operations.
- Improves **performance** and **disk lifespan** on SSDs and advanced format drives (4 KiB sectors).

Logical Volume Manager (LVM)

Use the Logical Volume Manager (LVM) system to create logical storage volumes as a layer on the physical storage. This storage system provides greater flexibility than using physical storage directly. LVM hides the hardware storage configuration from the software and enables you to resize volumes without stopping applications or unmounting file systems. LVM provides comprehensive command-line tools to manage storage.

Logical Volume Manager (LVM) is a storage abstraction layer in Linux that lets you manage disks with far more flexibility than traditional partitions. Instead of applications talking directly to physical disks, LVM inserts a smart layer in between.

This gives you:

- Dynamic resizing
- Storage pooling
- Snapshots
- Multi-disk aggregation
- Hardware-agnostic storage

It's the foundation of modern Linux storage design.

Logical Volume Manager Workflow

Creating LVM storage requires building structures in a logical workflow.

- Determine the physical devices used for creating physical volumes, and initialize these devices as LVM physical volumes.
- Create a volume group from multiple physical volumes.
- Create the logical volumes from the available space in the volume group.
- Format the logical volume with a file system and mount it, or activate it as swap space, or pass the raw volume to a database or storage server for advanced structures

The lvg and lvol Modules

The **lvg** and **lvol** modules support the creation of logical volumes, including the configuration of physical volumes, and volume groups. The **lvg** takes as parameters the block devices to configure as the back end physical volumes for the volume group.

1. The lvg Module (Volume Group Management)

The **lvg** module manages:

- Physical volumes (PVs)
- Volume groups (VGs)
- Physical extent size (PE size)

It can **create**, **extend**, or **remove** a volume group.

Important Parameters (lvg)

Parameter	Description
vg	Name of the volume group to create or manage.
pvs	Comma-separated list of block devices used as physical volumes.
pesize	Physical extent size (must be power of 2 or multiple of 128 KiB).
state	present (default) to create/extend, or absent to remove.

Example 1 — Create a Volume Group

Goal:

- VG name: vg1
- PV: /dev/vda1
- PE size: 32 (usually means 32 MiB)

```
``yaml
```

```
- name: Create a volume group
```

```
  lvg:
```

```
    vg: vg1
```

```
    pvs: /dev/vda1
```

```
    pesize: 32
```

```
``
```

✂ Example 2 — Extend a Volume Group

If `vg1` already exists with `/dev/vdb1`, this adds `/dev/vdc1`:

```
``yaml
```

```
- name: Extend volume group vg1
```

```
  lvg:
```

```
    vg: vg1
```

```
    pvs: /dev/vdb1,/dev/vdc1
```

```
``
```

This enlarges the VG by adding a new physical volume.

The lvol module

creates logical volumes, and supports the resizing and shrinking of those volumes, and the filesystems on top of them. This module also supports the creation of snapshots for the logical volumes.

The following table lists some of the parameters for the lvol module.

Parameter	Description
lv	Name of the logical volume.
vg	Parent volume group.
size	Size of the logical volume (e.g., 2g, 500m).
resizefs	yes to resize the filesystem along with the LV.

Parameter	Description
shrink	Allows shrinking the LV (disabled by default for safety).
snapshot	Name of the snapshot LV to create.
state	present to create/resize, absent to remove.

✂ Example — Create a Logical Volume

Goal:

- VG: vg1
- LV: lv1
- Size: 2 GB

```
```yaml
```

- name: Create a 2GB logical volume

lvol:

vg: vg1

lv: lv1

size: 2g

```
```
```

Quick Summary

✓ lvg manages:

- Physical volumes (PVs)
- Volume groups (VGs)
- PE size
- Adding/removing PVs

✓ lvol manages:

- Logical volumes (LVs)
- Resizing
- Filesystem resizing
- Snapshots

1. The filesystem Module

The **filesystem** module is used to:

- Create a new filesystem
- Resize an existing filesystem (grow only)

Supported filesystems include:

ext2, ext3, ext4, ext4dev, f2fs, lvm, xfs, vfat

Important Parameters (filesystem)

| Parameter | Description |
|-----------------|---|
| dev | Block device to format (e.g., /dev/vdb1, /dev/vg1/lv1). |
| fstype | Filesystem type (e.g., xfs, ext4, vfat). |
| resizefs | If yes, grow the filesystem to fill the device. |

Example — Create an XFS Filesystem

Goal:

- Filesystem: **XFS**
- Device: **/dev/vdb1**

``yaml

- name: Create an XFS filesystem

filesystem:

fstype: xfs

dev: /dev/vdb1

```

2. The mount Module

The **mount** module manages:

- Mounting devices
 - Updating `/etc/fstab`
 - Unmounting devices
 - NFS mounts
 - UUID-based mounts
-

Important Parameters (mount)

| Parameter | Description |
|---------------|---|
| path | Mount point directory (e.g., <code>/data</code>). |
| src | Device or NFS share to mount (e.g., <code>/dev/vdb1</code> , <code>UUID=xxxx</code> , <code>server:/share</code>). |
| fstype | Filesystem type (<code>xfs</code> , <code>ext4</code> , <code>nfs</code> , etc.). |
| opts | Mount options (defaults, <code>rw</code> , <code>noatime</code> , etc.). |
| state | mounted, unmounted, present, absent. |

State meanings:

- **mounted** → mount now + add to `/etc/fstab`
- **present** → only add to `/etc/fstab`, don't mount
- **unmounted** → unmount but keep in `fstab`
- **absent** → unmount + remove from `fstab`

Example — Mount Using UUID

Goal:

- Mount point: `/data`
- Device: UUID `a8063676-44dd-409a-b584-68be2c9f5570`
- Filesystem: `XFS`
- Add to `/etc/fstab`


```
``yaml
```

```
- name: Mount device with UUID
```

```
  mount:
```

```
    path: /data
```

```
    src: UUID=a8063676-44dd-409a-b584-68be2c9f5570
```

```
    fstype: xfs
```

```
    state: mounted
```

```
``
```

✂ Example — Mount an NFS Share

Goal:

- Mount /nfsshare
- NFS server: 172.25.250.100:/share

```
``yaml
```

```
- name: Mount NFS share
```

```
  mount:
```

```
    path: /nfsshare
```

```
    src: 172.25.250.100:/share
```

```
    fstype: nfs
```

```
    opts: defaults,nobootwait
```

```
    dump: 0
```

```
    passno: 2
```

```
    state: mounted
```

🌸 3. Configuring Swap with Ansible (No Direct Module)

Ansible has **no dedicated swap module**, so you must:

1. Create a VG (optional)
2. Create an LV for swap
3. Format it using mkswap
4. Enable it using swapon
5. Add to /etc/fstab

✂ Example — Create Swap Using LVM

```yaml

- name: Create VG for swap

lvg:

vg: vg\_swap

pvs: /dev/vdb1

- name: Create LV for swap

lvol:

vg: vg\_swap

lv: lv\_swap

size: 2g

- name: Format LV as swap

command: mkswap /dev/vg\_swap/lv\_swap

- name: Enable swap

command: swapon /dev/vg\_swap/lv\_swap

```

You can then add an `/etc/fstab` entry using the `mount` module with `fstype: swap`.

🧠 Quick Summary

✓ filesystem

Creates or grows filesystems.

✓ mount

Mounts devices and updates `/etc/fstab`.

✓ Swap

No module → use `lvg`, `lvvol`, `mkswap`, `swapon`.

✿ What Are Ansible Storage Facts?

When Ansible runs the **setup** module, it gathers hardware and system information from the managed host. For storage, the most important fact groups are:

- **ansible_devices** → All block devices + partitions
- **ansible_device_links** → IDs, UUIDs, labels
- **ansible_mounts** → Mounted filesystems
- **ansible_swaptotal_mb** → Total swap memory

You can retrieve all facts:

```
ansible webserver -m setup
```

Or filter specific facts:

```
ansible webserver -m setup -a "filter=ansible_devices"
```

✿ 2. ansible_devices — Block Devices & Partitions

This fact shows **all disks** on the system and their details:

- Disk size
- Sector size
- Whether it's virtual
- Partitions
- UUIDs
- Model/vendor
- Removable or not

Example:

```
ansible webserver -m setup -a "filter=ansible_devices"
```

Typical output structure:

```
ansible_devices:
```

```
  vda:
```

```
    size: "10.00 GB"
```

```
  partitions:
```

```
    vda1:
```

size: "10.00 GB"

uuid: "a8063676-44dd-409a-b584-68be2c9f5570"

vdb:

size: "5.00 GB"

partitions: {}

sr0:

size: "1024.00 MB"

removable: "1"

Why this matters:

- You can dynamically detect disks for LVM, partitioning, or filesystem tasks.
- You can automate storage without hardcoding device names.

✿ 3. `ansible_device_links` — IDs, UUIDs, Labels

This fact shows **symlinks** for devices:

- `/dev/disk/by-id`
- `/dev/disk/by-uuid`
- `/dev/disk/by-label`

Retrieve it:

```
ansible webserver -m setup -a "filter=ansible_device_links"
```

Example:

`ansible_device_links:`

ids:

sr0:

- ata-QEMU_DVD-ROM_QM00003

uuids:

vda1:

- a8063676-44dd-409a-b584-68be2c9f5570

Why this matters:

- UUIDs are **stable**, unlike `/dev/vda1` which can change.
- Best practice: mount using UUIDs.

✿ 4. `ansible_mounts` — Mounted Filesystems

Shows all currently mounted filesystems:

```
ansible webserver -m setup -a "filter=ansible_mounts"
```

Example:

`ansible_mounts:`

- device: `"/dev/vda1"`

mount: `"/"`

fstype: `"xfs"`

size_total: 10725863424

size_available: 9116598272

options: `"rw,seclabel,relatime,attr2,inode64,noquota"`

Why this matters:

- You can check if a filesystem is already mounted.
 - You can automate resizing or remounting.
-

✿ 5. `ansible_swaptotal_mb` — Swap Memory

Shows total swap in MB:

```
ansible_swaptotal_mb
```

Example usage:

- name: Activate swap LV

command: `swapon /dev/vgswap/lvswap`

when: `ansible_swaptotal_mb < 128`

Why this matters:

- You can automatically create swap if the system has too little.
-

Lab Scenario:

You are responsible for managing a set of web servers. A recommended practice for web server configuration is to store web server data on a separate partition or logical volume. You will write a playbook to:

- Manage partitions of the /dev/vdb device
- Manage a volume group named apache-vg for web server data
- Create two logical volumes named content-lv and logs-lv, both backed by the apache-vg volume group
- Create an XFS file system on both logical volumes
- Mount the content-lv logical volume at /var/www
- Mount the logs-lv logical volume at /var/log/httpd

If the storage requirements for the web server change, update the appropriate playbook variables and re-execute the playbook. The playbook should be idempotent.

PlayBook:

```
``yaml
```

```
---
```

```
- name: Storage layout for web servers
```

```
  hosts: webservers
```

```
  become: true
```

```
  vars:
```

```
    disk_device: /dev/vdb
```

```
    vg_name: apache-vg
```

```
    pv_partition: /dev/vdb1
```

```
    content_lv_name: content-lv
```

```
    content_lv_size: 5g
```

```
    content_mount: /var/www
```

```
    logs_lv_name: logs-lv
```

```
    logs_lv_size: 3g
```

```
    logs_mount: /var/log/httpd
```

fs_type: xfs

tasks:

- name: Create partition on /dev/vdb for LVM

ansible.builtin.parted:

device: "{{ disk_device }}"

number: 1

state: present

part_end: 100%

flags: [lvm]

unit: "%"

register: parted_result

- name: Create volume group for Apache data

community.general.lvg:

vg: "{{ vg_name }}"

pvs: "{{ pv_partition }}"

state: present

- name: Create logical volume for web content

community.general.lvol:

vg: "{{ vg_name }}"

lv: "{{ content_lv_name }}"

size: "{{ content_lv_size }}"

state: present

- name: Create logical volume for web logs

community.general.lvol:

vg: "{{ vg_name }}"

lv: "{{ logs_lv_name }}"

size: "{{ logs_lv_size }}"

state: present

- name: Create filesystem on content LV

ansible.builtin.filesystem:

fstype: "{{ fs_type }}"

dev: "/dev/{{ vg_name }}/{{ content_lv_name }}"

- name: Create filesystem on logs LV

ansible.builtin.filesystem:

fstype: "{{ fs_type }}"

dev: "/dev/{{ vg_name }}/{{ logs_lv_name }}"

- name: Ensure content mount point exists

ansible.builtin.file:

path: "{{ content_mount }}"

state: directory

mode: "0755"

- name: Ensure logs mount point exists

ansible.builtin.file:

path: "{{ logs_mount }}"

state: directory

mode: "0755"

- name: Mount content LV at {{ content_mount }}

ansible.builtin.mount:


```
path: "{{ content_mount }}"
src: "/dev/{{ vg_name }}/{{ content_lv_name }}"
fstype: "{{ fs_type }}"
state: mounted
```

- name: Mount logs LV at {{ logs_mount }}

ansible.builtin.mount:

```
path: "{{ logs_mount }}"
src: "/dev/{{ vg_name }}/{{ logs_lv_name }}"
fstype: "{{ fs_type }}"
state: mounted
```

...

To change layout later (sizes, mount points, VG name, disk), just adjust the section and rerun the playbook.